

Required Features

Crop Recommendations

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH
- Rainfall
- Soil_moisture
- Soil_type
- Sunlight_exposure
- Wind_speed

Crop Stress

- Soil_Moisture
- Ambient_Temperature
- Soil_Temperature
- Humidity
- Light_Intensity
- Soil_pH
- Nitrogen_Level
- Phosphorus_Level
- Potassium_Level

```
#7 Narrows down the crop recommendation features

crop_features = [
    "N",
    "P",
    "K",
    "temperature",
    "humidity",
    "ph",
    "rainfall",
    "soil_moisture",
    "soil_type",
    "sunlight_exposure",
    "wind_speed"
]

crop_target = "label"

missing_crop_columns = [col for col in crop_features + [crop_target] if col not in crop_df.columns]

if missing_crop_columns:
    print("Missing crop columns:")
    print(missing_crop_columns)
else:
    print("All crop recommendation columns are available.")

*** All crop recommendation columns are available.

#8 Narrows down the stress detection features

stress_features = [
    "Soil_Moisture",
    "Ambient_Temperature",
    "Soil_Temperature",
    "Humidity",
    "Light_Intensity",
    "Soil_pH",
    "Nitrogen_Level",
    "Phosphorus_Level",
    "Potassium_Level"
]

stress_target = "Plant_Health_Status"
```

Rule-based stress detection

Keep the trained crop recommendation model, but add warning rules.
Does not require new stress labels.

Ex.

If soil moisture is too low → drought stress warning

If temperature is too high → heat stress warning

If pH is too low or too high → pH stress warning

If N, P, or K is too low → nutrient stress warning

Create artificial stress labels

Create a new column called: environmental_stress

Ex:

0 = no stress

1 = stress

Stress could be defined based on thresholds.

Ex:

If soil moisture is below safe range → stress = 1

If pH is outside crop range → stress = 1

If temperature is too high/low → stress = 1

Otherwise → stress = 0

Train a second model:

sensor data + crop type = stress / no stress

These would be pseudo-labels. Labels created from rules, not labels measured from real crop health observations.

Possible Dataset

https://www.kaggle.com/datasets/ziya07/plant-health-data?utm_source=chatgpt.com

Tabular sensor dataset with readings like soil moisture, ambient temperature, soil temperature, humidity, light intensity, soil pH, nitrogen, phosphorus, potassium, chlorophyll content, and electrochemical signal.

It contains a real label column: **Plant_Health_Status**, with classes

- Healthy
- Moderate Stress
- High Stress

2025 Scientific Report

<https://www.nature.co/articles/s41598-025-26184-w>

Used this Kaggle dataset for plant stress prediction and notes that stress prediction was influenced by variables such as soil moisture, ambient temperature, soil temperature, and nitrogen level.

They used chlorophyll content and electrochemical signal, which we can avoid since we are using sensors to collect that data.

Model they used: SAM-L, which stands for a self-adaptive meta-learning approach. In simple terms, it is a machine learning framework designed to adjust better to changing environmental conditions. The paper combines this with an LSTM neural network, which is a type of neural network commonly used for sequential or time-based data. Their architecture used three LSTM layers, each with 256 neurons, and a softmax output layer for multi-class classification.

Models: LSTM, Decision Tree, Random Forest-Style → TinyML
Check the paper for the model they used

Training part: 80% of the rows for training

Testing part: 20% rows for validating

Small models + how accurate they are

By Tuesday 07/07/26

The most important features in the paper were **soil moisture, ambient temperature, soil temperature, and nitrogen level**. Soil moisture had the strongest influence on stress prediction

Models we can use: smaller neural network, decision tree, or random forest-style model converted to TinyML would be more practical.

For the prototype, we can use these columns:

- Soil_Moisture
- Ambient_Temperature
- Soil_Temperature
- Humidity
- Light_Intensity
- Soil_pH
- Nitrogen_Level
- Phosphorus_Level
- Potassium_Level

The dataset is described as a **simulated biosensor data**, not fully real-field collected data. We would be using a labeled simulated sensor dataset.

Train the model only using the mentioned features that match our available sensors, then later test or fine-tune the model using our own sensor readings.

Best Approach

Two datasets and two related models.

One crop recommendation dataset for crop suitability and the Plant-Health-Data dataset for stress detection.

Model 1: The crop recommendation model would predict the best crop based on N, P, K, temperature, humidity, pH, and rainfall.

Model 2: The stress model would predict whether the plant is Healthy, Moderate Stress, or High Stress using sensor-style data such as soil moisture, temperature, humidity, light, pH, and nutrients.

This gives the system two useful outputs: what crop is suitable and whether the current plant is under stress.

Models used

Model	Dataset	Purpose
Decision Tree	Crop Recommendation + Plant Health	Baseline Model
Small Random Forest-Style Model	Crop Recommendation + Plant Health	Better accuracy than one tree
Small LSTM	Mainly Plant Health	Compare with the paper

Model 1:

- 900 total rows
- 720 rows for training
- 180 for testing

9 crops were used.

Each crop had 100 rows

80 rows per crop for training

20 rows crop for testing

Used one-hot encoding for soil type

soil_type =

1 - Sandy

2 - Loamy

3 - Clay

Model 2:

- 1,200 total rows
- 960 for training
- 240 for testing

High stress: 500 rows

Moderate Stress: 401 rows

Healthy: 299 rows

Total: 1,200 rows

Training set:

High stress: 400 rows

Moderate Stress: 321 rows

Healthy: 239 rows

Total: 960 rows

Testing set:

High stress: 100rows

Moderate Stress: 80 rows

Healthy: 60 rows

Total: 240 rows

Task	Model	Accuracy	F1-Score	Notes
Crop Recommendation	Decision Tree	100%	1	In the crop recommendation Decision Tree, potassium was the most important feature. Nitrogen, phosphorus, humidity, temperature, and rainfall also contributed. Soil type, pH, soil moisture, sunlight exposure, and wind speed were not used by this specific tree.
Crop Recommendation	Small Random Forest	100%	1	In the crop recommendation Random Forest, humidity was the most important feature, followed by rainfall, phosphorus, potassium, nitrogen, and temperature. This suggests the model mainly relied on climate and nutrient conditions to

				recommend crops.
Crop Recommendation	Small LSTM			
Crop Stress	Decision Tree	100%	1	Soil moisture is the strongest indicator of plant stress in this dataset, and nitrogen level is the second strongest.
Crop Stress	Small Random Forest	97.92%	0.9791	
Crop Stress	Small LSTM			

Overall, the Decision Tree models were the most TinyML-friendly because they achieved very high accuracy with the smallest model sizes. The Random Forest models also performed strongly, but their model sizes were larger. For a microcontroller-based prototype, the Decision Tree is the best first option, while Random Forest can be used as a comparison model.

Run it on all the crops.

Run the new dataset.

Show why to choose one model over the other.

Show the accuracy between all the models.

100kb

New Dataset Trained

Used Decision Tree and Random Forest

Dataset distribution:

- Crop Type Distribution: 60% Wheat, 30% Maize, 10% Rice
- Label Imbalance: 70% Healthy crops, 30% Unhealthy crops

Crop Health Detection Results

Model	Accuracy	F1-Score	Model Size
Decision Tree	69.89	57.51	5.29kb
Random Forest	69.90	57.52	47.27kb

The model reached around 70% accuracy, but this result is misleading because the model mostly predicted the majority class, Healthy. It did not successfully detect Unhealthy samples.

Crop Stress Level Detection Results

Model	Accuracy	F1-Score	Model Size
Decision Tree	33.94	19.54	6.75kb
Random Forest	33.96	20.97	52.84kb

The stress-level models performed close to random guessing. This suggests that the selected features are not strongly related to the stress labels, or that the stress labels may not be well represented by these inputs.

Conclusion

The agriculture_dataset proves to be not reliable, since it depends a lot on image features. When training the models with the features that we will be using, related to our sensors, it shows low accuracy results. The dataset is also unevenly distributed, making it lean more toward “Healthy” crops. Stress-level detection performed close to random guessing at about 34% accuracy. This suggests that the selected non-image features are not enough to reliably predict crop health or stress in this dataset.

Project Overview

Deploy the Decision Tree Stress Model in the MSP430

Why? Because of it's size, and can be converted into simple if/else rules.

Deploy the Crop Recommendation model, heavier model, on the Raspberry Pi.
Given the soil and environmental conditions, what crop is suitable here?

Then it predicts one of the Texas-relevant crops:

- rice
- maize
- pomegranate
- grapes
- watermelon
- muskmelon
- apple

- orange
- cotton

The system uses two models. The first model is a crop recommendation model that runs on the Raspberry Pi and recommends a suitable crop based on soil and environmental conditions. The second model is a plant stress detection model that is small enough for TinyML and can run on the MSP430. The MSP430 reads sensor data and predicts whether the plant is Healthy, Moderate Stress, or High Stress. The Raspberry Pi receives the data, stores it, displays the results, and can also run larger comparison models like Random Forest.

Detect stress, then recommend a crop for the same planted field.

Mode 1: Crop planning

Recommend a suitable crop before planting.

Mode 2: Crop monitoring

Monitor the planted crop and detect stress.

Raspberry Pi

The Raspberry Pi can handle the crop recommendation model because this is not a real-time microcontroller task.

The user/farmer can enter field data or use stored sensor averages, and the Raspberry Pi says:

Recommended crop: cotton

This may only be done:

- once before planting
- once per field
- once per season

MSP430

The MSP430 should handle the stress detection model, because that is the real-time sensor task.

It can keep checking:

- soil moisture
- temperature
- humidity
- light

and predict:

- Healthy
- Moderate Stress
- High Stress

This may happen repeatedly:

- every few minutes
- every hour
- every day

Conclusion

The crop recommendation model and the stress detection model are used at different stages. The crop recommendation model is used before planting to suggest which crop is suitable for the field conditions. Once a crop is already planted, the crop recommendation model is no longer the main focus. At that point, the stress detection model monitors the crop using sensor readings and predicts whether the plant is Healthy, Moderate Stress, or High Stress. So the project supports both crop planning and crop monitoring

Friday Meeting 7/10/2026

What are the major factors that influence the crop stress, suitability, and what are the limitations, how can we propose something new.

Limitations

From the research papers that we found, some limitation of current IoT smart farming projects were:

- Sensors
- Power Supply

Contributions

Multi-Model IoT systems that learn from the data to monitor the crops.

Benefits

What are the benefits of our project compared to current research?

The Raspberry Pi can handle the heavier computation, have dataset that keeps constantly learning from it.

Isaac:

Test the crop recommendation on all the crops.

Theoretical foundations of how we determine crop suitability and how we determine crop stress. Make more data from the dataset based on the research; called Data Augmentation.

Skylar:

Big dataset

Missing following features:

- Sunlight exposure
- NPK Values

Crop Type Distribution: 60% Wheat, 30% Maize, 10% Rice

Label Imbalance: 70% Healthy crops, 30% Unhealthy crops

Presentation:

Background

Motivation

Proposed technique

Conclusion

Future Work